V-Twin — Installation, structure, configuration et déploiement reproductible

1. Résumé en une page

Contexte. V-Twin vise deux applications sur un noyau de simulation commun : une **sandbox de conception** (prioritaire pour le premier démonstrateur) et un **jumeau numérique** d'un moteur réel (livraison ultérieure). Le cas de référence est un moteur **Ricardo Hydra essence 86 × 86 mm**, contact **segment de feu – chemise**, usure liée au frottement/lubrification (périmètre produit).

Ce qui existe aujourd'hui (livré par #6, fermée) : un paquet Python numérique installable, une API HTTP locale en lecture seule, un écran Vue de démarrage, 41 tests Python + 6 tests frontend, des contrôles qualité et une CI sur Windows, macOS et Linux. **Aucun modèle physique, d'usure, runner, stockage, dashboard ni 3D n'est encore implémenté** (README).

Ce que ce livrable apporte :

Question	Réponse courte	Section
Que faut-il installer ?	uv 0.13.1, Node 24.18.1, npm 11.16.0 ; Python 3.13.16 est installé par uv	§3
Comment est organisé le code ?	Architecture hexagonale (domain / ports / engine / application / adapters), règles d'import vérifiées en CI	§4
Comment lancer ?	4 commandes, puis http://127.0.0.1:8000	§5
Comment configurer ?	Lancement : `VTWIN_HOST`, `VTWIN_PORT` (CLI > env > défaut), loopback obligatoire, pas de secret · Moteur : fichiers JSON validés par schéma	§6
Comment déployer ?	Application web **locale**, installation développeur ; **Docker non retenu** (justifié)	§7
Est-ce reproductible ?	Oui : vérifié depuis un clone propre + CI 3 OS	§8
Qu'est-ce qui manque ?	Worker, data directory, logs, sauvegarde/restauration → dépendent de #8, #40	§10

2. Avancement des critères d’acceptation de #41

#	Critère (#41)	Statut	Section
1	Prérequis, versions, structure des modules, commandes install/build/test/run, points d’entrée	**[OK]**	§3, §4, §5
2	`.env.example`, exemples dev/prod non sensibles, rôle des variables, validation, aucun secret	**[OK]** variables de lancement + fichiers de configuration moteur validés · **[PRÉVU]** data dir, logs, limites, jeton	§6
3	Déploiement retenu + procédure reproductible (Docker si justifié, sinon alternative expliquée)	**[OK]** local développeur ; Docker écarté avec justification	§7
4	Checkout propre : configuration, lancement, test d’un module critique, erreur de configuration	**[OK]** Linux local + CI 3 OS · **[PARTIEL]** postes réels de l’équipe	§8
5	Liens diagrammes, ADR, logs, sauvegarde/restauration, qualité ; distinguer disponible/prévu	**[OK]** liens et diagrammes · **[PRÉVU]** logs et sauvegarde/restauration	§9, §10

---

3. Prérequis et versions

Source : development.md — Supported toolchain.

Outil	Version	Fichier qui fait foi
CPython	**3.13.16** (série 3.13)	`.python-version` · `pyproject.toml` `requires-python = ">=3.13,<3.14"`
uv	**0.13.1** (imposée)	`pyproject.toml` `required-version = "==0.13.1"`
Node.js	**24.18.1**	`.node-version` · `.nvmrc`
npm	**11.16.0**	`web/package.json` `packageManager`
Dépendances Python	versions + hashes exacts	`uv.lock`
Dépendances web	résolution + intégrité exactes	`web/package-lock.json`

Trois niveaux de dépendances Python (`pyproject.toml`) :

Niveau	Contenu	Quand l'installer
Cœur (runtime numérique)	`numpy` ≥ 2.5.4, `scipy` ≥ 1.18.1	Toujours ; suffit pour `vtwin check-numerics`
Extra `api`	`fastapi` 0.146, `uvicorn` 0.54	Pour `vtwin serve`
Groupe `dev`	pytest, pytest-cov, ruff, mypy, import-linter, httpx, psutil 7.2.2	Développement, tests, inventaire machine

Node/npm = build et développement uniquement (Vue 3.5.44, TypeScript 5.9, Vite 8.3). À l'exécution, l'interface compilée est servie par le processus Python (architecture — Local deployment). **Aucun GPU ni CUDA requis** en V1 (ADR 0002).

Installer les outils : uv via ses instructions officielles (version 0.13.1) ; Node 24.18.1 via nodejs.org ou un gestionnaire qui lit `.nvmrc` ; puis `npm install --global npm@11.16.0`.

Postes cibles de l'équipe

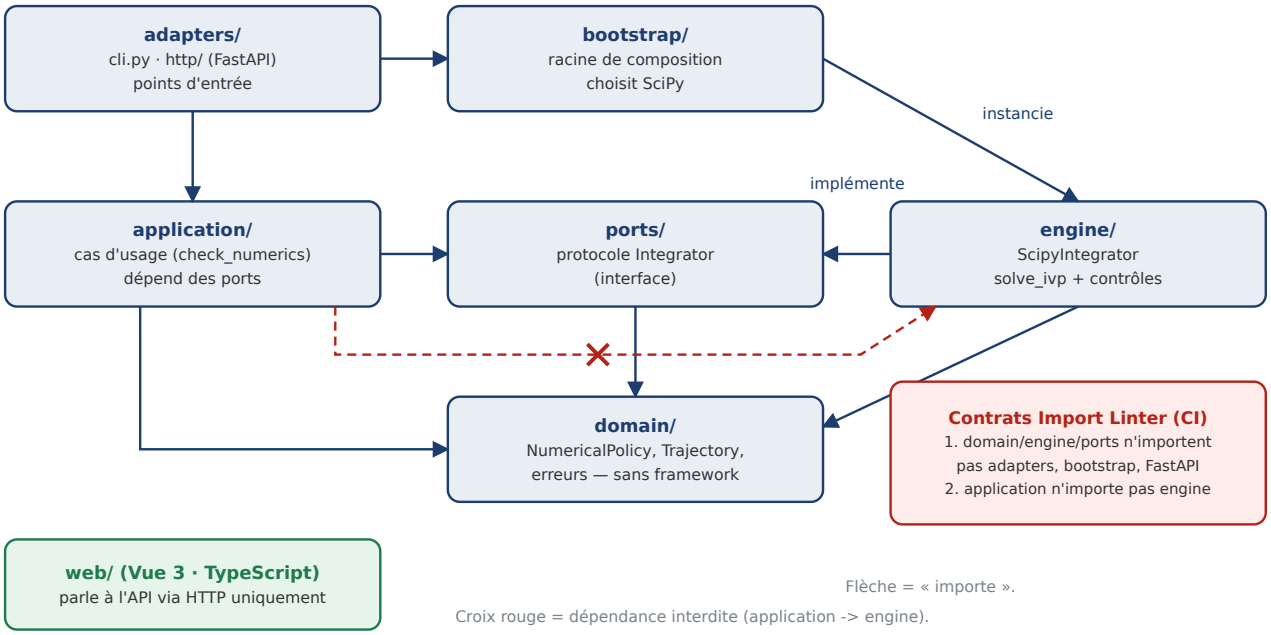
Source : registre matériel du 24/10/2026 et `hardware-inventory.json` (valeurs **déclarées**, non mesurées).

Membre / profil	CPU	RAM	Graphique	Rôle
Pacien · `pacien-16gb-igpu`	i5-1135G7	16 Go	Iris Xe intégré	**Profil de référence** (GPU intégré)
Divin · `divin-16gb-gtx1050ti`	i5-7300HQ	16 Go	GTX 1050 Ti	**Profil de référence** (GPU dédié)
Raphael · `raphael-m4-16gb`	Apple M4	16 Go	non précisé	profil distinct (macOS ARM)
Mathieu · `mathieu-m4-16gb`	Apple M4	16 Go	non précisé	profil distinct (macOS ARM)
Adam · `adam-32gb-rtx5060`	i7-13620H	32 Go	RTX 5060 Laptop	profil distinct

---

4. Structure des modules et points d'entrée

Sources : development.md — Module responsibilities, architecture — Module structure.



Couches du paquet `vtwin` et règles d'import vérifiées en CI

Chemin	Responsabilité implémentée
`src/vtwin/domain/`	Politique numérique, trajectoires, erreurs — sans framework
`src/vtwin/ports/`	Protocole d'intégrateur remplaçable (contrats moteur : voir §6.4)
`src/vtwin/engine/`	Adaptateur ODE SciPy avec contrôles de finitude, forme et complétude
`src/vtwin/application/`	Contrôle analytique via le port
`src/vtwin/bootstrap/`	Racine de composition : sélection explicite de SciPy
`src/vtwin/adapters/`	Points d'entrée CLI et HTTP FastAPI (optionnel)
`web/src/api/`, `web/src/App.vue`	Client API versionné + écran de connexion, sans 3D
`tests/`, `web/src/*.test.ts`	Tests numériques, adaptateurs, frontend
`scripts/`, `.github/workflows/`	Rapport qualité, smoke test serveur réel, inventaire machine, CI

Règles vérifiées à chaque CI (contrats Import Linter) : (1) le cœur `domain/engine/ports` n'importe ni `adapters`, ni `bootstrap`, ni FastAPI/uvicorn ; (2) `application` dépend des ports, jamais d'`engine`. C'est ce qui permettra de remplacer le moteur Python par du C++ sans toucher à l'API (ADR 0002).

Points d'entrée

Commande	Rôle	Source
`vtwin serve`	API locale + interface compilée	`adapters/cli.py`
`vtwin serve --no-web`	API seule, pour le serveur Vite	idem

Commande	Rôle	Source
`vtwin check-numerics`	Contrôle analytique (`y' = -y` comparé à `exp(-t)`), sans serveur	`application/foundation.py`
`python -m vtwin`	Équivalent de `vtwin`	`__main__.py`
`npm --prefix web run dev`	Serveur de développement Vite	`web/package.json`

Routes HTTP exposées (lecture seule)

Source : `adapters/http/app.py`.

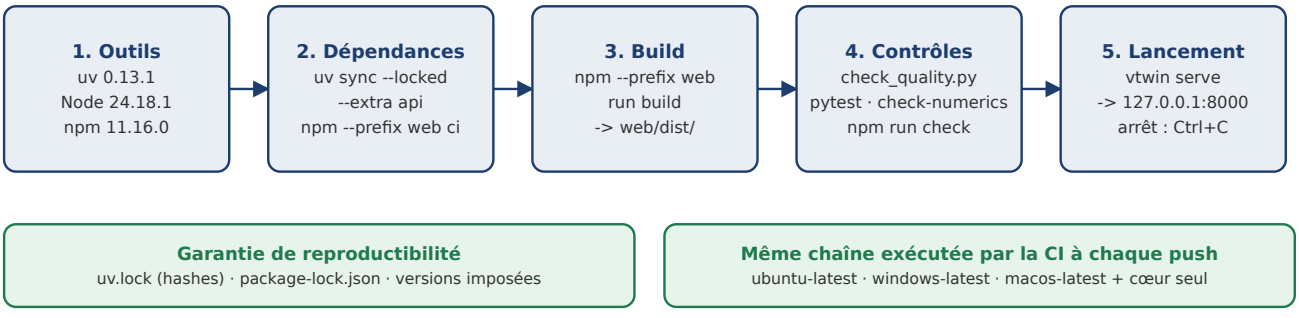
Route	Réponse vérifiée le 26/10
`GET /health`	`{"status":"ok","version":"0.1.0"}`
`GET /api/v1/capabilities`	`stage: "foundation"`, `models: []`, `simulation_available: false`, `viewer_3d_available: false`
`GET /api/v1/openapi.json`	Schéma OpenAPI
`/`	Interface compilée (HTTP 200)

Aucune route de soumission de calcul ni de lecture de résultats : elles attendent les contrôles de session/erreurs/logs de #40 (architecture — Errors and trust boundaries).

---

5. Commandes : installer, construire, tester, lancer

Toutes les commandes s'exécutent **depuis la racine du dépôt**, à l'identique sous PowerShell, zsh ou bash (development.md — Install and run).



Python 3.13.16 est installé automatiquement par uv (.python-version).
Node/npm servent uniquement au build : à l'exécution, l'UI compilée est servie par Python.

Source : docs/development.md · .github/workflows/ci.yml

Procédure d'installation reproductible

5.1 Installation et lancement

```
git clone https://github.com/V-Twin-EIP/V-Twin.git
cd V-Twin
uv sync --locked --extra api     # dépendances Python (uv.lock)
npm --prefix web ci              # dépendances web (package-lock.json)
npm --prefix web run build       # interface compilée -> web/dist/
uv run --locked --extra api vtwin serve
```

Ouvrir http://127.0.0.1:8000 ; arrêt par `Ctrl+C`. `--locked` et `npm ci` **refusent** toute divergence avec les lockfiles au lieu de les modifier : c'est la base de la reproductibilité.

5.2 Mode développement frontend

```
# Terminal 1 : API seule
uv run --locked --extra api vtwin serve --no-web
# Terminal 2 : Vite
npm --prefix web run dev
```

Vite relaie `/api` et `/health` vers `127.0.0.1:8000` (`vite.config.ts`).

5.3 Qualité et tests

```
# Python : ruff, mypy strict, import-linter
uv run --locked --extra api python scripts/check_quality.py
# Tests + couverture
uv run --locked --extra api pytest --cov=vtwin
# Module critique numérique
uv run --locked --extra api vtwin check-numerics
# Web : lint, format, Vitest, build
npm --prefix web run check
# Paquet : wheel + sdist
uv build --no-sources
# Vrai serveur, vraies routes
uv run --locked --extra api python scripts/smoke_http.py
```

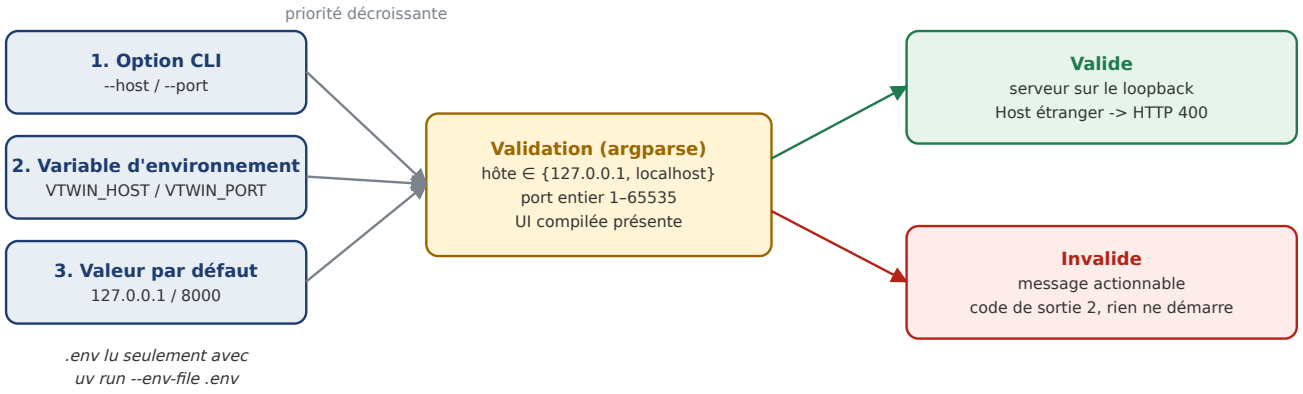
Rapports produits dans `artifacts/` (ignoré par Git) et téléversés par la CI pour chaque OS.

5.4 Mode « dashboard sans 3D »

Exigence produit : le parcours configuration/calcul/graphiques/comparaison/export doit fonctionner **sans initialiser la 3D** (périmètre, ADR 0001). **Aujourd'hui** le socle ne charge aucune bibliothèque 3D : tout lancement est déjà sans 3D. Le choix explicite dashboard/3D arrivera avec #24 **[PRÉVU]**.

---

6. Configuration



Source : src/vtwin/adapters/cli.py · http/app.py

Résolution et validation de la configuration de lancement

6.1 Variables actuelles

Source : `.env.example`, `cli.py`.

```
# Optional local launch configuration; contains no secrets.
VTWIN_HOST=127.0.0.1
VTWIN_PORT=8000
```

Variable	Option CLI	Défaut	Rôle	Validation
`VTWIN_HOST`	`--host`	`127.0.0.1`	Adresse d'écoute	Uniquement `127.0.0.1` ou `localhost`
`VTWIN_PORT`	`--port`	`8000`	Port d'écoute	Entier de 1 à 65535

- **Priorité :** option CLI > variable d'environnement > défaut.
- **`.env` jamais chargé implicitement :** `cp .env.example .env` (PowerShell : `Copy-Item`), puis `uv run --env-file .env --locked --extra api vtwin serve`.
- **Aucun secret :** `.env` et `.env.*` sont exclus de Git sauf `.env.example` (`.gitignore`).
- **Sécurité initiale :** écoute sur loopback uniquement, `TrustedHostMiddleware` (en-tête `Host` étranger → HTTP 400), pas de CORS générique.

6.2 Exemples développement / utilisation locale

En V1, la « production » est **l'usage local par un ingénieur sur son poste** (§7).

	Développement	Utilisation locale (« prod » V1)
`.env`	`VTWIN_HOST=127.0.0.1` `VTWIN_PORT=8000`	`VTWIN_HOST=127.0.0.1` `VTWIN_PORT=8080` (port libre au choix)
Lancement	`vtwin serve --no-web` + `npm --prefix web run dev`	`npm --prefix web run build` puis `vtwin serve`

	Développement	Utilisation locale (« prod » V1)
Interface	Vite, rechargement à chaud	`web/dist/` servi par Python, même origine
Contrainte	le port doit correspondre au proxy Vite	aucune

(Les commandes `vtwin` s'exécutent via `uv run --env-file .env --locked --extra api`.)

6.3 Variables prévues

D'après l'architecture, seront ajoutées **quand leur module existera** (le guide interdit les réglages « inertes ») : répertoire de données utilisateur hors Git, origines de développement autorisées, niveau/destination des logs, limites de file et d'exécution, jeton de session local **jamais persisté ni journalisé [PRÉVU — #8, #40]**. Les limites seront fixées par mesure (#39), pas déduites des 16 Go.

6.4 Configuration du moteur (fichiers de conception)

La configuration de **lancement** (§6.1) est distincte de la configuration du **moteur simulé**. Celle-ci est un fichier JSON versionné, décrit par un schéma, avec pour chaque valeur son unité SI, son statut (`sourced`, `derived`, `assumed`, `tbd`) et sa source (contrats de données).

Fichier	Rôle
`contrats-v0.1/design-configuration.schema.json`	Schéma de la configuration de conception (JSON Schema 2020-12)
`exemples/conf-a-hydra-nominal.json`	Référence Ricardo Hydra 86 × 86 mm, taux 10,5 (valeurs sourcées uniquement)
`exemples/conf-b-viscosite.json`	Variante : seule la loi de viscosité change
`exemples/invalides/`	10 configurations erronées qui doivent être rejetées

Ces fichiers ne contiennent **aucun secret** et peuvent être versionnés. Vérification :

```
uv run --with jsonschema==4.26.0 python docs/livrables/contrats-v0.1/valider.py
```

Une erreur indique le champ exact et la règle violée, par exemple un alésage saisi en millimètres :

```
$.engine.bore.unit: 'm' was expected
```

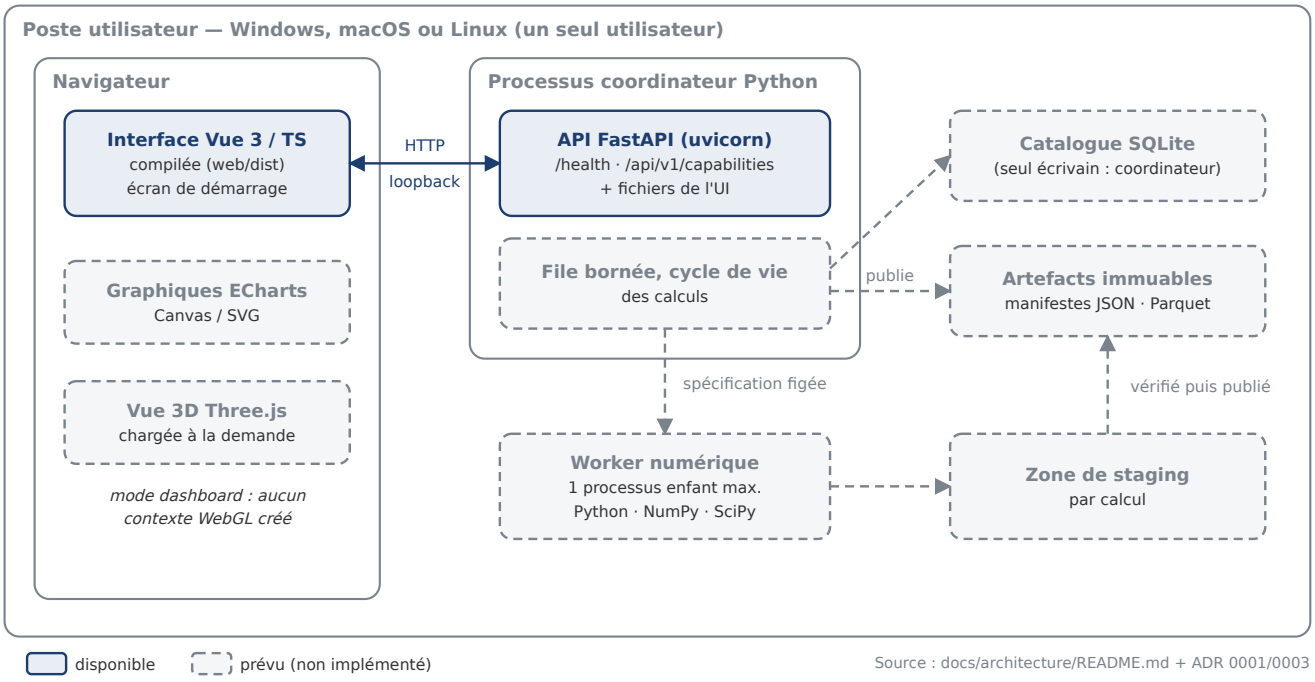
Résultat du 26/10/2026 : **12/12 conformes** (2 configurations acceptées, 10 rejetées).

---

7. Déploiement retenu

7.1 Décision

Application web locale, installation développeur par commandes documentées, sur Windows, macOS et Linux ; pas d'installateur en un clic pour V1. Décision prise en équipe (périmètre — Decision record, ADR 0001).



Topologie de déploiement local : actuel et prévu

Un seul processus coordinateur sert l'API et l'interface sur la même origine loopback. En cible, il pilotera **au plus un worker numérique** en processus enfant, un catalogue **SQLite** et des artefacts **JSON/Parquet** immuables (ADR 0003).

7.2 Pourquoi pas Docker

Le plan Technical Track cite Docker comme *une* option possible, à choisir « après que la stack et les besoins d'exploitation sont connus ». Comparaison :

Critère	Docker	Installation locale uv + npm (retenue)
Reproductibilité	Image figée	Lockfiles avec hashes + `--locked` / `npm ci` + versions d'outils imposées
Windows / macOS	Docker Desktop, couche VM	Natif sur les 3 OS, vérifié en CI à chaque push
Mesures de performance (protocole)	La VM ajoute de la mémoire et fausse le KPI `peak_sampled_app_rss_bytes` sur les postes 16 Go	Mesure directe des processus applicatifs
Navigateur / 3D	Reste sur l'hôte : aucun gain	Identique

Critère	Docker	Installation locale uv + npm (retenue)
Services à orchestrer	Aucun : pas de base serveur, pas de broker (ADR 0003)	Un seul processus

Conclusion : Docker n'apporte pas de reproductibilité supplémentaire pour une application mono-utilisateur sans service externe, et gênerait les mesures de performance. Il sera **réévalué** si un service serveur est introduit pour le jumeau (InfluxDB/TimescaleDB : #32, #43).

7.3 Procédure reproductible

La procédure est la suite du §5.1, **exécutée automatiquement** par la CI à chaque push et pull request : installation verrouillée → inventaire d'environnement → qualité → tests → build Python → build web → **lancement réel du serveur** → téléversement des rapports. Actions épinglées par SHA, permissions en lecture seule, aucun secret requis.

---

8. Vérification depuis un checkout propre (26/10/2026)

8.1 Vérification locale

Clone neuf de `main` au commit `eeb3c91`, Linux x86_64, **uv 0.13.1 · Python 3.13.16 · Node 24.18.1 · npm 11.16.0**.

Étape	Commande	Résultat
Installation Python	`uv sync --locked --extra api`	**[OK]**
Installation web	`npm --prefix web ci`	**[OK]**
Qualité Python	`scripts/check_quality.py`	**[OK]** lint, format, types, boundaries : 4/4 PASS
Tests Python	`pytest --cov=vtwin`	**[OK] 41 tests passés**, couverture **87 %** (lignes + branches)
Module critique : intégrateur	`vtwin check-numerics`	**[OK]** erreur max **1,53e-10** pour une tolérance de **1e-7** (518 évaluations)
Frontend	`npm --prefix web run check`	**[OK]** lint, format, **6 tests** Vitest, build (JS 64,8 kB / 25,6 kB gzip)
Paquet	`uv build --no-sources`	**[OK]** `vtwin-0.1.0-py3-none-any.whl` + `.tar.gz`
Lancement réel	`scripts/smoke_http.py`	**[OK]** UI, `/health` et `/api/v1/capabilities` sur un vrai serveur
Configuration `.env`	`VTWIN_PORT=8123` + `--env-file .env`	**[OK]** serveur sur 8123, `/` → HTTP 200
Hôte étranger	`Host: evil.example`	**[OK]** rejeté, HTTP 400

Le contrôle numérique valide le **contrat logiciel** de l'intégrateur, pas un modèle physique ni l'usure (development.md — Quality and tests).

8.2 Erreurs de configuration

Chaque erreur s'arrête **avant** le démarrage, avec un message actionnable et le **code de sortie 2** :

Situation	Message
`--host 0.0.0.0` ou `VTWIN_HOST=0.0.0.0`	`Only loopback hosts are supported.`
`VTWIN_PORT=70000` ou `--port 0`	`Port must be between 1 and 65535.`
Interface non compilée	`Build the UI first with npm --prefix web run build, or use --no-web.`
Extra `api` absent (lu dans le code)	`The HTTP adapter requires the api extra: uv sync --locked --extra api`

Ces cas sont aussi testés automatiquement (`tests/test_cli.py`).

8.3 Vérification CI multi-OS

Run n°12 sur `eeb3c91`, 26/10/2026 :

Job	Résultat	Durée
Foundation (ubuntu-latest)	**[OK]**	44 s
Foundation (windows-latest)	**[OK]**	1 min 49 s
Foundation (macos-latest)	**[OK]**	2 min 14 s
Cœur seul : wheel non éditable, sans FastAPI	**[OK]**	11 s

Le run initial de #6 (39164388597, 18/10) était déjà vert sur les 4 jobs.

8.4 Ce qui reste à vérifier [PARTIEL]

La CI prouve la portabilité sur des **machines hébergées**, pas sur les **postes de l'équipe** (benchmarks/README.md). Prochaine étape : sur chaque poste, installer selon §5.1 et enregistrer l'environnement :

```
uv run --locked --extra api python scripts/performance_inventory.py --label divin-16gb-gtx1050ti --output artifacts/performance/divin-16gb-gtx1050ti.json
```

Cette commande collecte l'environnement, **pas** une performance. Les objectifs utilisateur — **≤ 60 s par calcul** et **< 1 s** pour ouvrir un résultat ou changer une courbe — ne sont **pas encore mesurés** (protocole, #39, #46).

---

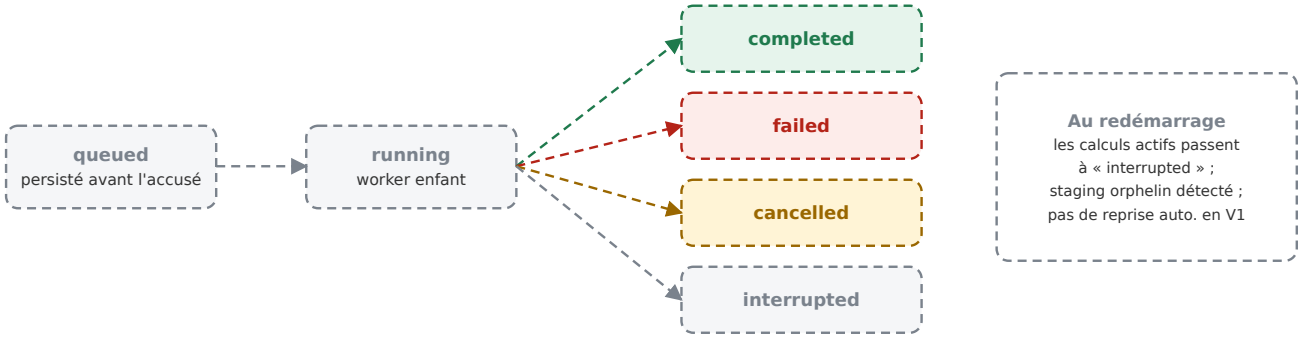
9. Documents liés

Sujet	Document	Statut
Architecture V1 : composants, UML, flux, déploiement	docs/architecture/README.md	**[OK]**
ADR 0001 — application locale et frontière UI	adr/0001	**[OK]**
ADR 0002 — moteur Python de référence, politique numérique	adr/0002	**[OK]**
ADR 0003 — exécution locale, stockage, sauvegarde/restauration	adr/0003	**[OK]** décision · **[PRÉVU]** implémentation
Guide développeur (référence anglaise)	docs/development.md	**[OK]**
Contribution	CONTRIBUTING.md	**[OK]**
Rapports qualité par OS	artefacts `foundation-<os>` de la CI	**[OK]**
KPI et protocole de performance	docs/performance.md, benchmarks/README.md	**[OK]** protocole · **[PRÉVU]** mesures
Périmètre produit	docs/product-scope.md	**[OK]**
Scénarios de validation de la sandbox (6 scénarios)	docs/validation/	**[OK]** spécification · **[PRÉVU]** exécution
Plan Technical Track (objectif O2)	docs/technical-track.md	**[OK]**
Contrats de données et configuration moteur	M1.3-contrats-de-donnees.md	**[OK]** schéma et exemples · **[PRÉVU]** intégration au code
Erreurs, logs structurés, jeton de session	#40	**[PRÉVU]**

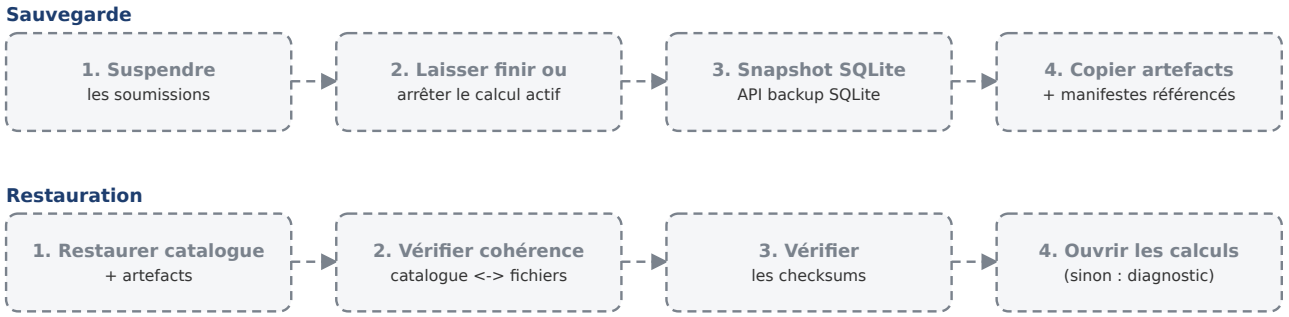
---

10. Ce qui reste à faire dans #41

Ces procédures sont **décidées** dans l'architecture mais **ne peuvent pas être exercées** tant que le code correspondant n'existe pas.



Cycle de vie d'un calcul (prévu)



Procédure décidée (ADR 0003) — à implémenter et exercer quand la persistance existera (#8, #40, #41).

Sauvegarde et restauration (prévu)

Élément	Décision documentée	Dépend de
Lancement/arrêt coordinateur + worker	1 coordinateur, ≤ 1 worker enfant (`spawn`), file bornée persistée	#8
Répertoire de données utilisateur	Par utilisateur, hors Git	#8
Interruption / réconciliation	Au redémarrage : actifs → `interrupted`, staging orphelin détecté, pas de reprise auto.	#8
Sauvegarde	Suspendre → laisser finir/arrêter → backup SQLite → copier artefacts/manifestes	#8, #40
Restauration	Vérifier cohérence catalogue/fichiers et checksums avant d'ouvrir	#8, #40
Export	Adaptateurs d'export explicites (JSON/CSV)	#8, #24
Logs	Structurés : run ID, composant, sévérité, code d'erreur ; sans jeton ni secret	#40
Limites de file / exécution	Fixées par mesure sur les postes 16 Go	#39
Installation sur postes réels	Divin et Pacien d'abord, mode dashboard	#39